```
# ----------------- HEAVY XGBoost TUNING CELL (paste & run) -----------------
!pip install xgboost --quiet

import os, numpy as np, pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedKFold, RandomizedSearchCV, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
import joblib, warnings, time, math
warnings.filterwarnings("ignore")

OUTDIR = Path("analysis_outputs"); OUTDIR.mkdir(exist_ok=True)

def save_fig(fig, name):
    p = OUTDIR / name
    fig.savefig(p, bbox_inches="tight", dpi=200)
    print("Saved:", p)
    return p

# 0) Load CSV (adjust path if needed)
CSV_PATH = "/content/SURVEY~1.CSV"
if not os.path.exists(CSV_PATH):
    files = [f for f in os.listdir(".") if f.endswith(".csv")]
    if len(files) == 1:
        CSV_PATH = files[0]
    else:
        raise FileNotFoundError("CSV not found; upload or set CSV_PATH.")
df = pd.read_csv(CSV_PATH)
print("Loaded shape:", df.shape)

# normalize column names
def normalize(col):
    c = str(col).strip().lower()
    c = c.replace(' ', '_').replace('-', '_')
    c = ''.join(ch for ch in c if (ch.isalnum() or ch == '_'))
    return c
df.columns = [normalize(c) for c in df.columns]

# drop agreed columns and remove rows missing target
drop_cols = ["timestamp","study_year","socioeconomic_background","assignment_submission_timing","use_of_time_management","effect_of
df = df.drop(columns=[c for c in drop_cols if c in df.columns], errors='ignore')
df = df.dropna(subset=['grade_outcome']).reset_index(drop=True)

# map to binary target
def map_binary2(g):
    g = str(g).strip()
    try:
        if g.lower().startswith('below'):
            return 0
        num = float(g.split()[0])
        return 1 if num >= 3.0 else 0
    except:
        if 'below' in g.lower() or g.startswith('2.'):
            return 0
        return 1
df['high_gpa'] = df['grade_outcome'].apply(map_binary2)
print("Target distribution (0=Low,1=High):")
print(df['high_gpa'].value_counts())

# Prepare X,y
y = df['high_gpa'].astype(int)
X = df.drop(columns=['grade_outcome','high_gpa'])

# coerce numeric-like columns
for c in X.columns:
    if X[c].dtype == object:
        s = X[c].astype(str).str.replace(',', '').str.strip()
        if s.str.match(r'^[0-9]+(\.[0-9]+)?$').mean() > 0.6:
            X[c] = pd.to_numeric(s, errors='coerce')
```

```
num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = [c for c in X.columns if c not in num_cols]
print("num cols:", num_cols)
print("cat sample:", cat_cols[:15])

# Preprocessor
preproc_tree = ColumnTransformer([
    ("num", "passthrough", num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
], remainder="drop")

# Baselines (keep them unchanged)
pipe_lr = Pipeline([("pre", preproc_tree), ("clf", LogisticRegression(max_iter=2000))])
pipe_dt = Pipeline([("pre", preproc_tree), ("clf", DecisionTreeClassifier(random_state=42))])

# Quick baseline CV (optional)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
print("Baseline CV (LR & DT) - this is just for reference")
for name, pipe in [("LogisticRegression", pipe_lr), ("DecisionTree", pipe_dt)]:
    scores = cross_val_score(pipe, X, y, cv=cv, scoring='accuracy', n_jobs=-1)
    print(f"{name} CV accuracy: {scores.mean():.4f} ± {scores.std():.4f}")

# Encode X once for faster XGBoost training
X_enc = preproc_tree.fit_transform(X)
try:
    feat_names = preproc_tree.get_feature_names_out()
except Exception:
    feat_names = [f"f{i}" for i in range(X_enc.shape[1])]
print("Encoded shape:", X_enc.shape)

# Compute scale_pos_weight for XGBoost if class imbalance exists
pos = int(y.sum()); neg = int(len(y)-pos)
scale_pos_weight = neg/pos if pos > 0 else 1.0
print("scale_pos_weight:", scale_pos_weight)

# -------------- RandomizedSearchCV (aggressive) ----------------
# Wide distributions for exploration. n_iter set high for more coverage.
xgb_base = XGBClassifier(objective="binary:logistic", use_label_encoder=False, eval_metric="logloss", random_state=42, n_jobs=4)

param_dist = {
    "n_estimators": [200, 400, 800, 1200, 1600, 2000],
    "max_depth": [3, 4, 6, 8, 10, 12],
    "learning_rate": [0.001, 0.005, 0.01, 0.02, 0.03, 0.05, 0.1],
    "subsample": [0.5, 0.6, 0.7, 0.8, 1.0],
    "colsample_bytree": [0.4, 0.6, 0.8, 1.0],
    "gamma": [0, 0.01, 0.05, 0.1, 0.3],
    "reg_lambda": [0, 1, 2, 5],
    "scale_pos_weight": [scale_pos_weight, 1.0]  # try balancing and not balancing
}

# RandomizedSearchCV over pre-encoded arrays using sklearn wrapper to avoid repeated preprocessing
# We'll wrap XGB in a pipeline that uses passthrough preproc (already applied), so we can pass arrays directly.
# For convenience, we use X_enc as features; RandomizedSearchCV accepts arrays.
rs = RandomizedSearchCV(
    estimator = xgb_base,
    param_distributions = param_dist,
    n_iter = 120,               # aggressive search
    scoring = "accuracy",
    cv = cv,
    random_state = 42,
    verbose = 2,
    n_jobs = -1,
    return_train_score = False
)

print("\nStarting RandomizedSearchCV (this may take a while)...")
t0 = time.time()
rs.fit(X_enc, y)
t1 = time.time()
print(f"RandomizedSearchCV done in {(t1-t0)/60:.1f} minutes")

print("Best params found:")
print(rs.best_params_)
print("Best CV accuracy (rs):", rs.best_score_)

# -------------- Final training with early stopping ----------------
best_params = rs.best_params_.copy()
# We'll use a train/validation/test split to allow early stopping
```

```
# We'll use a train/validation/test split to allow early stopping
X_temp, X_test, y_temp, y_test = train_test_split(X_enc, y, test_size=0.25, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.2, stratify=y_temp, random_state=42)
print("Train/Val/Test sizes:", X_train.shape[0], X_val.shape[0], X_test.shape[0])

# Build final XGBoost with best params
final_xgb = XGBClassifier(objective="binary:logistic",
                          use_label_encoder=False,
                          eval_metric="logloss",
                          random_state=42,
                          n_jobs=4,
                          **best_params)

# Fit with early stopping
print("\nFitting final XGBoost with early stopping (may take a few minutes)...")
final_xgb.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=50)

# Evaluate on holdout test set
y_pred_test = final_xgb.predict(X_test)
acc_test = accuracy_score(y_test, y_pred_test)
f1_test = f1_score(y_test, y_pred_test)
print("\nFINAL XGBoost (tuned) HOLDOUT METRICS")
print("Accuracy (holdout):", round(acc_test,4))
print("F1 (holdout):", round(f1_test,4))
print(classification_report(y_test, y_pred_test))

# Save confusion matrix
cm = confusion_matrix(y_test, y_pred_test)
fig, ax = plt.subplots(figsize=(5,4))
ax.imshow(cm, cmap=plt.cm.Blues, interpolation='nearest')
ax.set_title("Confusion Matrix: XGBoost_final")
ax.set_xticks([0,1]); ax.set_yticks([0,1])
ax.set_xticklabels(['Low','High']); ax.set_yticklabels(['Low','High'])
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, cm[i,j], ha='center', va='center', color='white' if cm[i,j] > cm.max()/2 else 'black')
save_fig(fig, "confusion_matrix_xgb_final.png")
plt.close(fig)

# -------------- Baseline holdout (unchanged) ----------------
# Fit baseline pipelines on the same preprocessor + full training data (to be comparable)
pipe_lr = Pipeline([("pre", preproc_tree), ("clf", LogisticRegression(max_iter=2000))])
pipe_dt = Pipeline([("pre", preproc_tree), ("clf", DecisionTreeClassifier(random_state=42))])
# Fit on the entire training+val (X_temp) mapped via preprocessor:
X_full_for_pipes = X_temp  # X_temp is already pre-encoded; to use pipes we need raw X. So fit them on full raw X.
pipe_lr.fit(X, y)   # baseline fit on full data (untuned per your requirement)
pipe_dt.fit(X, y)

# Evaluate LR & DT on X_test: need to transform X_test back to raw input — we have raw X (not encoded)
# We still have raw X split earlier? We'll re-split raw X similarly to encoded split to ensure same holdout
X_raw = X.values if not isinstance(X, pd.DataFrame) else X
X_temp_raw, X_test_raw, y_temp_raw, y_test_raw = train_test_split(X_raw, y, test_size=0.25, stratify=y, random_state=42)
# Fit baselines on full raw X (already done), now predict on X_test_raw
y_pred_lr = pipe_lr.predict(X_test_raw)
y_pred_dt = pipe_dt.predict(X_test_raw)

print("\nLogisticRegression HOLDOUT (baseline, unchanged)")
print("Accuracy:", round(accuracy_score(y_test_raw, y_pred_lr),4))
print(classification_report(y_test_raw, y_pred_lr))

fig, ax = plt.subplots(figsize=(5,4))
cm_lr = confusion_matrix(y_test_raw, y_pred_lr)
ax.imshow(cm_lr, cmap=plt.cm.Blues, interpolation='nearest')
ax.set_title("Confusion Matrix: LogisticRegression_baseline")
for i in range(cm_lr.shape[0]):
    for j in range(cm_lr.shape[1]):
        ax.text(j, i, cm_lr[i,j], ha='center', va='center', color='white' if cm_lr[i,j] > cm_lr.max()/2 else 'black')
save_fig(fig, "confusion_matrix_LogisticRegression_baseline.png")
plt.close(fig)

print("\nDecisionTree HOLDOUT (baseline, unchanged)")
print("Accuracy:", round(accuracy_score(y_test_raw, y_pred_dt),4))
print(classification_report(y_test_raw, y_pred_dt))
fig, ax = plt.subplots(figsize=(5,4))
cm_dt = confusion_matrix(y_test_raw, y_pred_dt)
ax.imshow(cm_dt, cmap=plt.cm.Blues, interpolation='nearest')
ax.set_title("Confusion Matrix: DecisionTree_baseline")
for i in range(cm_dt.shape[0]):
```

```
    for j in range(cm_dt.shape[1]):
        ax.text(j, i, cm_dt[i,j], ha='center', va='center', color='white' if cm_dt[i,j] > cm_dt.max()/2 else 'black')
save_fig(fig, "confusion_matrix_DecisionTree_baseline.png")
plt.close(fig)

# -------------- Save final pipeline & importances ----------------
# Wrap preprocessor + final_xgb so it's easy to load later
final_pipe = make_pipeline(preproc_tree, final_xgb)
joblib.dump(final_pipe, OUTDIR / "xgb_heavy_tuned_pipeline.joblib")
print("Saved final pipeline to:", OUTDIR / "xgb_heavy_tuned_pipeline.joblib")

# Feature importances
try:
    importances = final_xgb.feature_importances_
    fi = pd.DataFrame({"feature": feat_names, "importance": importances}).sort_values("importance", ascending=False).head(30)
    fi.to_csv(OUTDIR/"feature_importances_xgb_heavy.csv", index=False)
    fig, ax = plt.subplots(figsize=(8,6))
    ax.barh(fi["feature"][::-1], fi["importance"][::-1])
    ax.set_title("Top features: XGBoost (heavy tuned)")
    save_fig(fig, "feature_importances_xgb_heavy.png")
    plt.close(fig)
    display(fi.head(15))
except Exception as e:
    print("Feature importance save failed:", e)

print("\nHEAVY TUNING RUN COMPLETE. Files in analysis_outputs/. Please paste the printed holdout Accuracy and classification_report
# ---------------------------------------------------------------------------
```

```
Loaded shape: (451, 16)
Target distribution (0=Low,1=High):
high_gpa
1    273
0    168
Name: count, dtype: int64
num cols: []
cat sample: ['socio_economic_background', 'assignment_delay_frequency', 'procrastination_reasons', 'last_minute_exam_preparation',
Baseline CV (LR & DT) - this is just for reference
LogisticRegression CV accuracy: 0.7846 ± 0.0184
DecisionTree CV accuracy: 0.8095 ± 0.0084
Encoded shape: (441, 150)
scale_pos_weight: 0.6153846153846154

Starting RandomizedSearchCV (this may take a while)...
Fitting 5 folds for each of 120 candidates, totalling 600 fits
RandomizedSearchCV done in 2.9 minutes
Best params found:
{'subsample': 0.6, 'scale_pos_weight': 0.6153846153846154, 'reg_lambda': 5, 'n_estimators': 200, 'max_depth': 10, 'learning_rate':
Best CV accuracy (rs): 0.832150153217569
Train/Val/Test sizes: 264 66 111

Fitting final XGBoost with early stopping (may take a few minutes)...
[0] validation_0-logloss:0.68855
[50]    validation_0-logloss:0.54264
[100]   validation_0-logloss:0.49247
[150]   validation_0-logloss:0.46131
[199]   validation_0-logloss:0.44609

FINAL XGBoost (tuned) HOLDOUT METRICS
Accuracy (holdout): 0.8559
F1 (holdout): 0.8689
              precision    recall  f1-score   support

           0       0.72      1.00      0.84        42
           1       1.00      0.77      0.87        69

    accuracy                           0.86       111
   macro avg       0.86      0.88      0.85       111
weighted avg       0.90      0.86      0.86       111

Saved: analysis_outputs/confusion_matrix_xgb_final.png

LogisticRegression HOLDOUT (baseline, unchanged)
Accuracy: 0.9279
              precision    recall  f1-score   support

           0       0.87      0.95      0.91        42
           1       0.97      0.91      0.94        69

    accuracy                           0.93       111
   macro avg       0.92      0.93      0.92       111
weighted avg       0.93      0.93      0.93       111

Saved: analysis_outputs/confusion_matrix_LogisticRegression_baseline.png

DecisionTree HOLDOUT (baseline, unchanged)
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        42
           1       1.00      1.00      1.00        69

    accuracy                           1.00       111
   macro avg       1.00      1.00      1.00       111
weighted avg       1.00      1.00      1.00       111

Saved: analysis_outputs/confusion_matrix_DecisionTree_baseline.png
Saved final pipeline to: analysis_outputs/xgb_heavy_tuned_pipeline.joblib
Saved: analysis_outputs/feature_importances_xgb_heavy.png
```

feature	importance